

Multi-Source Candidate Data Transformer — Technical Design

1 PIPELINE BREAKDOWN



Parse — four independent parsers (CSV, ATS JSON, GitHub REST API, recruiter notes TXT) each produce a *RawCandidate* with a confidence score and extraction method per field. Parser failures are caught and logged; the pipeline never crashes on a single bad source.

Normalize — phones → E.164, emails → lowercase, dates → YYYY-MM, countries → ISO-3166 alpha-2; unknown values stored as null, never guessed. **Merge** — group by primary email; source priority resolves scalar conflicts; lists are unioned. **Score** — per-field confidence averaged to overall_confidence; +0.05 boost when ≥2 sources agree. **Validate** — checks candidate_id, email presence, full_name, confidence in [0,1]; logs warnings. **Project** — runtime config reshapes output without mutating the canonical record.

2 CANONICAL OUTPUT SCHEMA & FORMATS

Field	Type / Format
candidate_id	SHA-256(email)[:16] hex — deterministic
full_name	string
emails	string[] — lowercase
phones	string[] — E.164 e.g. +14088670001
location	city, region, country (ISO alpha-2 e.g. US)
links	linkedin, github, portfolio
years_experience	number null
skills	[[name, confidence, sources[]]]
experience	[[company, title, start, end]] — dates YYYY-MM
education	[[institution, degree, field, end_year]]
provenance	[[field, source, method]]
overall_confidence	float [0.0 – 1.0]

3 MERGE & CONFLICT RESOLUTION

- Identity key:** normalized primary email. No email → isolated UUID group.
- Source priority (low → high):** recruiter_notes < ats_json < recruiter_csv < github
- Scalar fields** (name, headline, title): highest-priority source with a non-blank value wins. Sources processed in ascending priority order so the last write wins.
- List fields** (emails, phones): union across all sources, deduplicated after normalization.
- Skills:** deduplicated by lowercase canonical name. If the same skill appears in ≥2 sources, it receives a +0.05 confidence boost (capped at 1.0).
- Experience:** deduplicated by company name (case-insensitive); higher-priority title replaces lower-priority one.
- Education:** deduplicated by institution name.

4 RUNTIME CONFIG — PROJECTION LAYER

- Core pipeline is config-unaware.** It always produces the full CanonicalCandidate. The Projector reads the config and builds a new Map — the canonical record is never mutated.
- Field selection:** fields[] lists output keys. Omit entirely for the full profile.
- Rename / remap:** from sets the source path; path sets the output key name.
- Path expressions:** direct field (full_name), list index (emails[0]), sub-field of every element (skills[].name).
- On missing:** "null" outputs null, "omit" skips the key, "error" throws if required.
- Toggles:** include_confidence, include_provenance (both default true).
- Per-field normalize:** optional pass — email, phone, date, country, skill aliases.

5 EDGE CASES

Case	Handling
Candidate with no email (e.g. Carol in CSV)	Assigned a UUID group key so the record is not lost and is not falsely merged with anyone else.
Invalid / fictional phone number	libphonenumber rejects it; stored as null. Downstream consumers can trust every non-null phone is valid E.164.
Same candidate with different titles across sources	Source priority order ensures the highest-reliability source (GitHub > CSV > ATS > notes) wins. No data is discarded — provenance records all sources.
Country as free text ("United States", "canada", "US")	Static lookup map normalizes all variants to ISO alpha-2. Unrecognized values are stored as-is rather than dropped.
Skill aliases (k8s, js, ml, nodejs)	canonicalizeSkill() maps aliases before deduplication so "k8s" and "Kubernetes" count as the same skill and correctly boost each other's confidence.

⑥ DELIBERATELY LEFT OUT

- **LinkedIn source** — requires OAuth 2.0 flow; not feasible within time constraints. Noted in README.
- **PDF / DOCX resume parsing** — needs a dedicated extraction library; complex and out of scope for this assignment.
- **Fuzzy name-based identity matching** — high false-positive risk (e.g. "John Smith"). Email is a reliable unique key; name matching is explicitly excluded.
- **GitHub personal access token** — 60 unauthenticated requests per hour is sufficient for a demo. Token management adds complexity with no benefit here.
- **Async / streaming processing** — a synchronous single-threaded pipeline is correct, simple, and testable at this scale. Async would add complexity for no observable benefit.